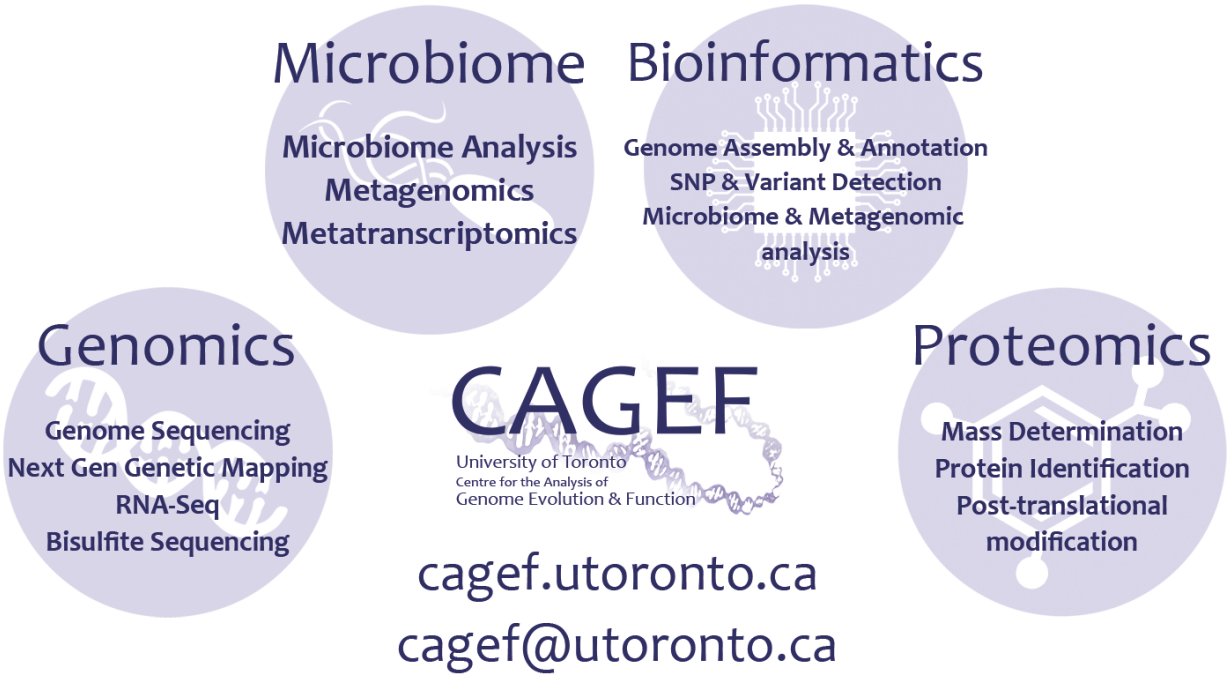


cagef.utoronto.ca
cagef@utoronto.ca

CSB1021HF LEC0131

FUNDAMENTALS OF GENOMIC DATA SCIENCETEST

0.0.0 Module 2: Assembly and annotation with Galaxy

0.1.0 About Fundamentals of Genomic Data Science

Fundamentals of Genomic Data Science is brought to you by the **Centre for the Analysis of Genome Evolution & Function (CAGEF)** bioinformatics training initiative. This course was developed based on feedback on the needs and interests of the Department of Cell & Systems Biology and the Department of Ecology and Evolutionary Biology.

The structure of this course is a "code-along", hands-on style! A few hours prior to each lecture, materials will be made available for download at QUERCUS (https://q.utoronto.ca/). The teaching materials will consist of a weekly PDF that you can use to follow along with the instructor along with any datasets that you'll need to complete the module. This learning approach will allow you to spend the time coding and not taking notes!

As we go along, there will be some in-class challenge questions for you to solve. Post lecture assessments will also be available for each module, building upon the concepts learned in class (see syllabus for grading scheme and percentages of the final mark).

0.1.1 Where is this course going?

We'll take a blank slate approach here to learning genomic data science and assume you know nothing about programming or working directly with next generation sequencing data. From the beginning of this course to the end we want to guide you from potential scenarios like:

- You don't know what to do with a set of raw sequencing files fresh from a facility like CAGEF.

University of Toronto Centre for the Analysis of Genome Evolution & Function (cagef.utoronto.ca) Licensed under CC BY-NC-SA

- You've been handed a legacy pipeline to analyse your data or maintain for the lab, but you don't know what it runs or how.
- You plan on generating high-throughput data but there are no bioinformaticians around to help you out.

and get you to the point where you can:

- Recognize the basic tools in sequence analysis.
- Plan and write your own data analysis pipelines.
- Explain your data analysis methods to labmates, supervisors, and other colleagues.

0.1.2 How do we get there?

In the first half of this course, we'll focus on how to generate analysis pipelines using the Galaxy platform – a user-friendly graphical interface that provides access to common sequence analysis tools. After we are comfortable with these tools, we'll look at life through the lens of a command-line interface. It is here that we will learn the basics of file manipulation and how to program scripts that can carry out multiple tasks for us. From there we'll revisit tools from the first half and learn skills to make your data analysis life easier.

0.2.0 Goals of the module

1. Gain familiarity with the Galaxy interface.
2. Learn how to perform quality control of raw sequencing reads with Galaxy.
3. Learn how to perform genome assembly and annotation with Galaxy.
4. Learn how to save and extract data and workflows with Galaxy.

0.3.0 Pre-class modules with Coursera

Each week we strongly encourage you to complete the assigned Coursera modules and/or readings ***before*** class. These are meant to provide you with sufficient background material on each week's module so that we can focus on the act of "doing" something with that data rather than spend a lot of time on the origins of it. You'll find a section outlining the next set of Coursera modules and readings ***at the end*** of each module.

0.3.1 Go to www.coursera.org and sign up for an account with your e-mail.

0.3.2 Search the following courses and enroll to audit each course (audit):

- Command Line Tools for Genomic Data Science, Johns Hopkins University.

0.4.0 Setting up your working directory

We suggest that you create a new directory (folder) for this course directly off your root directory called "**FGDS**". Working from your root directory is not necessary, but it will make some of the aspects of the course a little easier to manage. For MacOS users, we suggest you create this as a subfolder in your **user** directory.

0.4.1 Within this directory, create another directory called "**Module2**". This is where we will store the data used in this week's module.

0.4.2 Create a subdirectory called "**downloads**" to store the initial files as we download them before decompressing and working with them in later steps.

University of Toronto Centre for the Analysis of Genome Evolution & Function (cagef.utoronto.ca) Licensed under CC BY-NC-SA

1.0.0 Logging into Galaxy

This week we'll being working with the Galaxy platform and becoming familiar with how to navigate around it. There are a few ways you can use galaxy between using an account over at https://usegalaxy.ca/, or setting up your own server instance or trying to set up your own cloud version with Amazon Web services. Today we'll mainly cover the first option by working with an instance maintained by the Digital Research Alliance of Canada, Calcul Québec, and the Université de Sherbrooke.

Why are we using Galaxy? As we mentioned in the class slides, just as with the world of biological science, it's important that our experiments and results are reproducible. While we often encounter detailed biological methods in manuscripts, we might forget that the same applies to our data analysis.

Although we may take this idea for granted, it's important to remember that software tools and packages can undergo revision! Running the same set of data between *different versions* of tools can result in separate outputs. Furthermore, if you don't list your parameter choices in detail, it can make your results harder to interpret and replicate.

While tracking all these details can be quite important, approaching software tools can also be daunting! We use **Galaxy** as a good way to ease into these software packages, learn to manipulate their parameters, and simplify the process of connecting their output to other tools. The graphical user interface greatly decreases the initial energy barrier to learning these common bioinformatic tools.

As we'll see later in class, you can also save these steps into workflows (pipelines) which you can use to repeat your analyses on different data sets or to tweak the parameters for a single dataset.

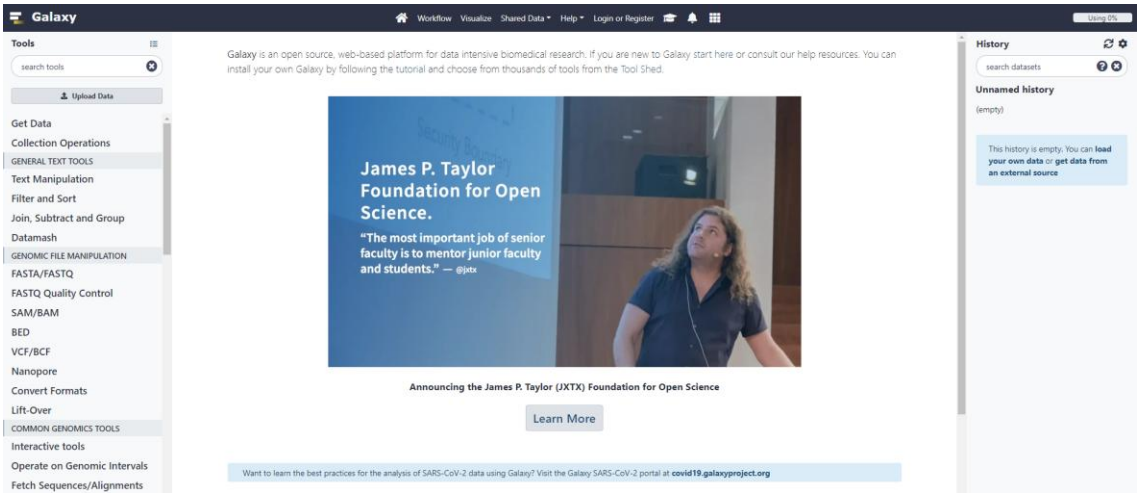
1.1.0 Galaxy Registration with the Galaxy Main resource

After working with galaxy in this course you may wish to continue exploring it on the publicly available galaxy servers worldwide. There are a few instances such as the original one at https://usegalaxy.org where you'll find a number of tutorials, workflows, and datasets available. The Canadian galaxy instance should closely mirror, however, most of the tools available on the main instance. After we're done with our work in this course you could choose to use this as an alternate workspace to carry out some of your own pipelines although these may have longer queue times than the Canadian version.

1.1.1 Go to https://usegalaxy.org.

1.1.2 Select "Login or Register" located at the top of the page. Fill in all required fields and then submit.

University of Toronto Centre for the Analysis of Genome Evolution & Function (cagef.utoronto.ca) Licensed under CC BY-NC-SA



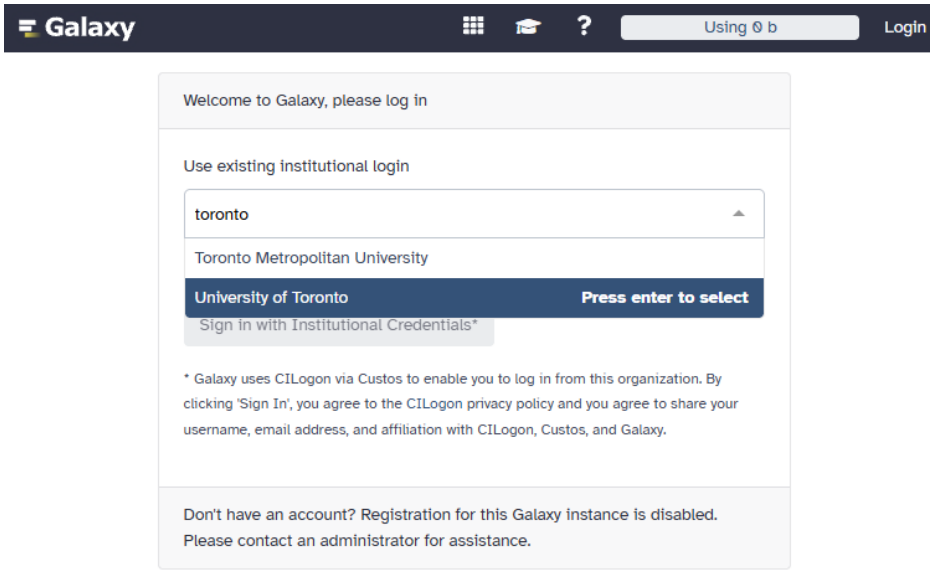
1.1.3 Verify your account by selecting the link in your email after it arrives.

The downside to using the online platform is that it can sometimes take several hours or even days for your analysis to get to the front of the queue

1.2.0 Running on the usegalaxy.ca instance

The https://usegalaxy.ca instance offers computing infrastructure to Canadian researchers. It is a cloud-based platform service that is built on Digital Research Alliance of Canada (previously known as Compute Canada) resources. While having a Compute Canada Role Identifier (CCRI) can grant you easy access to these and additional resources, it is not necessary to use the galaxy instance. Instead, you can use your University of Toronto login information (usually 2FA-based) to access the servers.

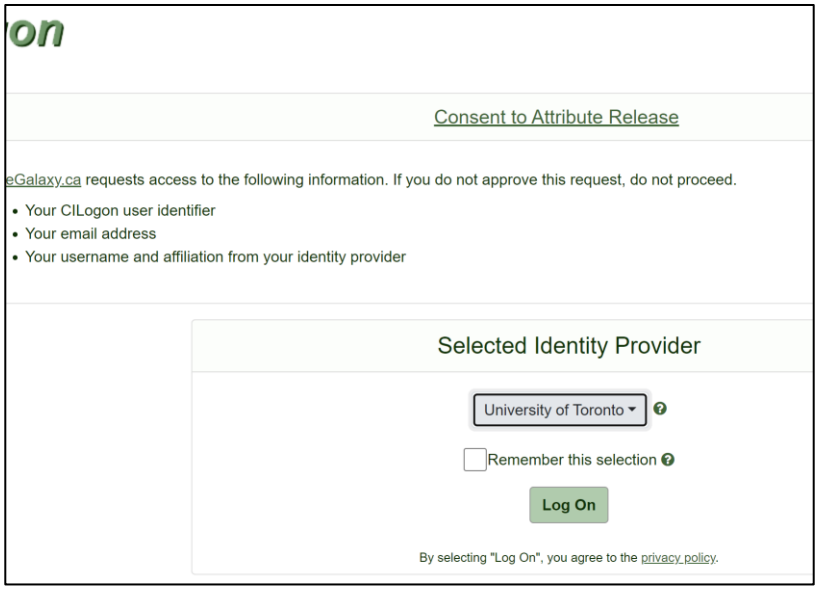
1.2.1 Login using your CILogon credentials – these will either take you to your uToronto page (with 2FA) or a google login page



1.2.1 Complete login using your University of Toronto credentials

University of Toronto Centre for the Analysis of Genome Evolution & Function (cagef.utoronto.ca) Licensed under CC BY-NC-SA





Once you are logged in, you'll be brought to the Galaxy homescreen. It is from here that we'll be doing all of our activities.

Do I need to use University of Toronto credentials? You aren't required to use a Canadian University's credential system and can, instead, use something like a Google account to log in. However, using a university credential will currently grant you access to 500Gb of storage space while a Google account may give you less space.

Given the size of many raw sequencing files and the output from processes like sequence alignment, it's much better if you stick with university-based credentials!

1.3.0 The Galaxy interface has four main quadrants



University of Toronto Centre for the Analysis of Genome Evolution & Function (cagef.utoronto.ca) Licensed under CC BY-NC-SA

As you look at your galaxy home screen there are four quadrants to identify. Knowing the location and meaning of these will make navigating and following lecture instructions a little easier.

1.3.1 The Galaxy *application screen* is the main portion of your interface. It is centred on the screen and defaults to 80% of the application interface. Here you can view results, datasets, and tool options as we navigate through our pipelines.

1.3.2 The Galaxy *menu bar* is located at the top of the page. This menu can link you to a deep source of learning materials as well as allow you to change user settings etc.

1.3.3 The Galaxy *activity pane* is located on the left side of the screen. This is most useful when working on the **Home** *application screen*. Looking at this **activity menu,** you'll find access to bioinformatics tools, datasets, histories and more! Clicking on the second icon of the **activity menu**, the **Tools** icon, will make available all the analysis tools sorted into categories by function. You can also use the *Search* bar to find tools by name.

1.3.4 The Galaxy *History pane* on the far right shows us the order of tools and applications that we have used. This can be used to see what you've done but also to put a pipeline together! We'll be referring to this often throughout our instructions.

Now that we've familiarized ourselves with the general interface of Galaxy, we can attempt to use some of the more popular tools and workflows. First, however, we need to import some data.



University of Toronto Centre for the Analysis of Genome Evolution & Function (cagef.utoronto.ca) Licensed under CC BY-NC-SA

2.0.0 Importing data files in Galaxy

Before we can begin working with any data, we need to import it from somewhere. We can choose to upload data from local sources, from outside servers, or from within the Galaxy server itself. We'll walk through some helpful habits and instructions that will make it easier to import data for your data analysis

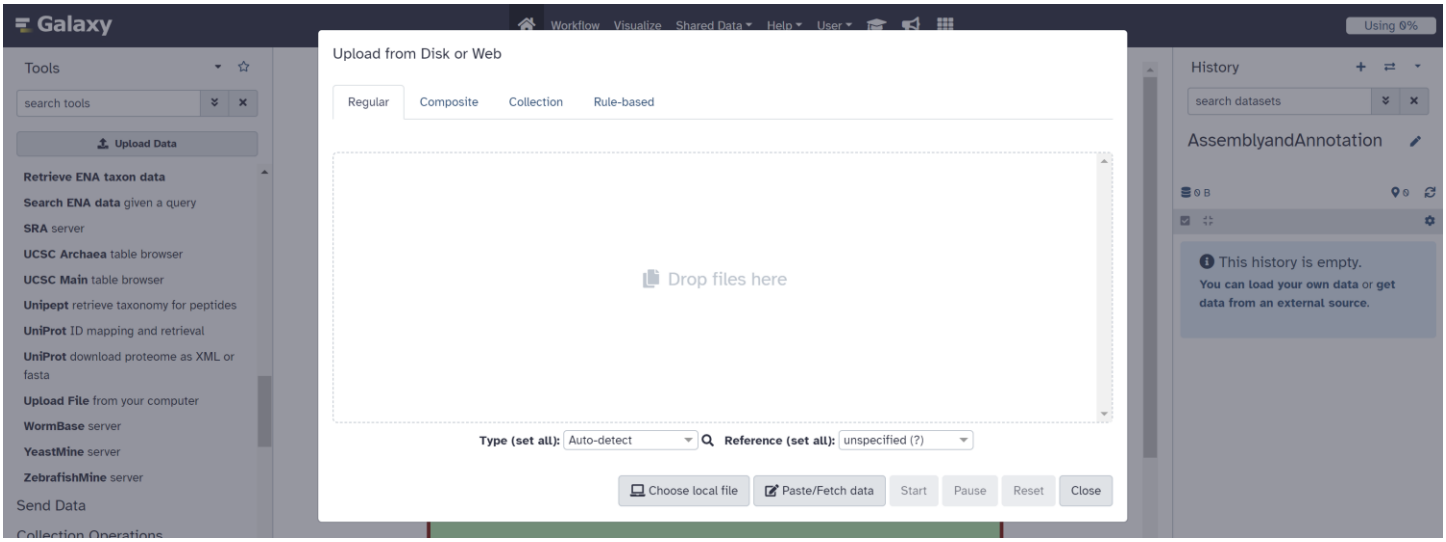
2.1.0 Creating a new history

2.1.1 In the *History pane*, choose the **+** symbol in the top right to create a new history.

2.2.0 Retrieving local data to analyse on Galaxy

For a typical analysis on Galaxy, you will upload the required data to the server using the **Get Data** tool from the *tool pane*. This option should be the top tool in your *tool pane* and clicking on it you'll see many options! This includes downloading and extracting FASTQ reads directly from the NCBI SRA but we won't do that today. Alternatively, use the **Upload** icon, which should be just above the **Tools** icon on the Tools pane.

2.2.1 For now, we'll focus on **Get Data > Upload File from your computer.** Clicking on this option will bring up the following window:

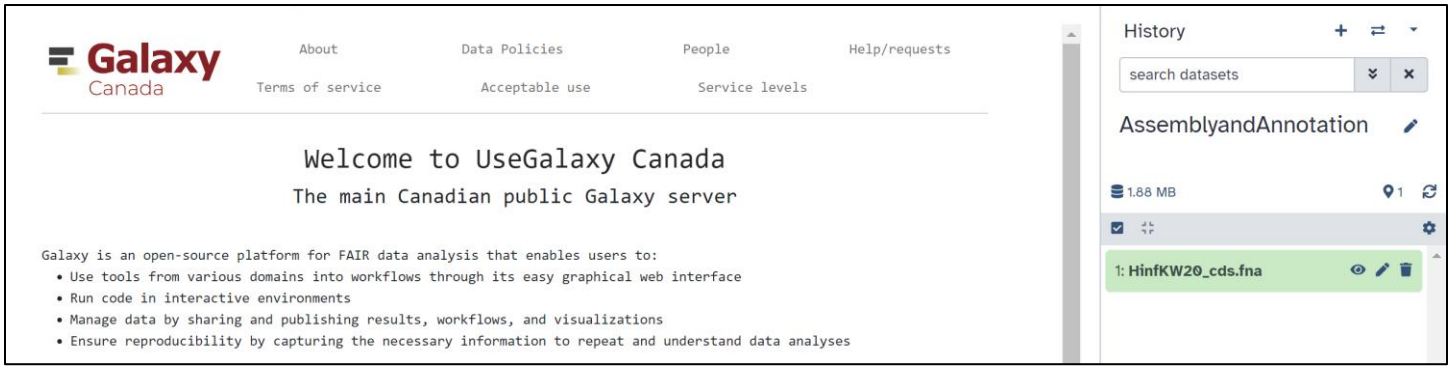


2.2.2 The data you will need for this module is the raw fastq reads for *H. influenzae* str. KW20. Unfortunately, upload and download speeds can be painstakingly slow for large files. Therefore, we will instead do a mock upload with a much smaller file (**HinfKW20_cds.fna**) to provide an example of how this would work. Use the **Choose local file** button to upload this file data from last week.

2.2.3 Hit the **Start** button to begin the upload. Note that you can queue multiple files for upload!

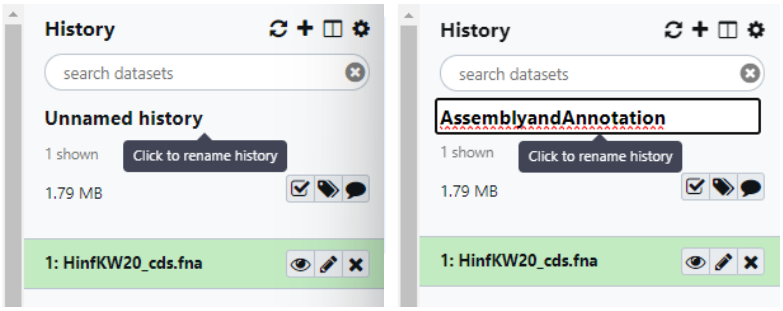
2.2.4 While the file is uploading, you'll see it in the *History pane* first as a grey entry with a small clock icon, then as a yellowish entry with a spinning circle icon. When the operation is complete, the entry will turn a green colour.

University of Toronto Centre for the Analysis of Genome Evolution & Function (cagef.utoronto.ca) Licensed under CC BY-NC-SA



Notice that the entry has a number beside it? Each entry name in your *History pane* will be formatted as **#:history_entry** so that you can keep track of the order for your operations as well. As you create more entries, they will be listed in *descending* order.

2.2.5 Rename your current history from **Unnamed history** to **AssemblyandAnnotation**. Click/select the name area in the History tab and then rename it by typing the name and pressing the Enter/Return key.



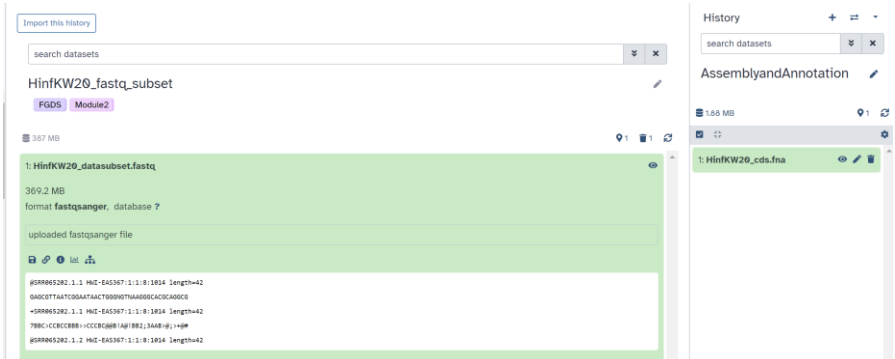
2.3.0 Retrieving shared histories to analyse on Galaxy

While the above instructions are a useful example, we will still need the fastq file with our sequencing reads to perform assembly and annotation. A subset file of only 2,000,000 reads has been uploaded to the usegalaxy.ca server for you to access and load to your own history:

2.3.1. From the Galaxy *activity pane* choose **Histories** and choose the **Shared with Me** tab from the Application pane.

2.3.2 From the dataset list, choose **HinfKW20_fastq_subset** and from the dropdown menu (triangle) choose **View**. You will notice that this includes two tags: FGDS and Module2

2.3.3 Once viewing the history, you can choose **HinfKW20_datasubset.fastq** and it will expand to show you additional information:



University of Toronto Centre for the Analysis of Genome Evolution & Function (cagef.utoronto.ca) Licensed under CC BY-NC-SA

2.3.3 In the upper left corner of the application pane, click on **Import this history**. This will allow you to copy this history and its data into your Galaxy.

2.3.4 In the pop-up window, name the title for the history to **HinfKW20_fastq_subset**.

2.3.5 Choose the option to **Copy only the active, non-deleted datasets** before clicking on the **Copy History** button.

The process for adding shared datasets to your history will be similar for a regular Galaxy server. However, our class data has been specially shared using your Galaxy-associated email accounts. You may find shared histories through the **Public Histories** tab as well

2.4.0 Add an imported history to **your** own history

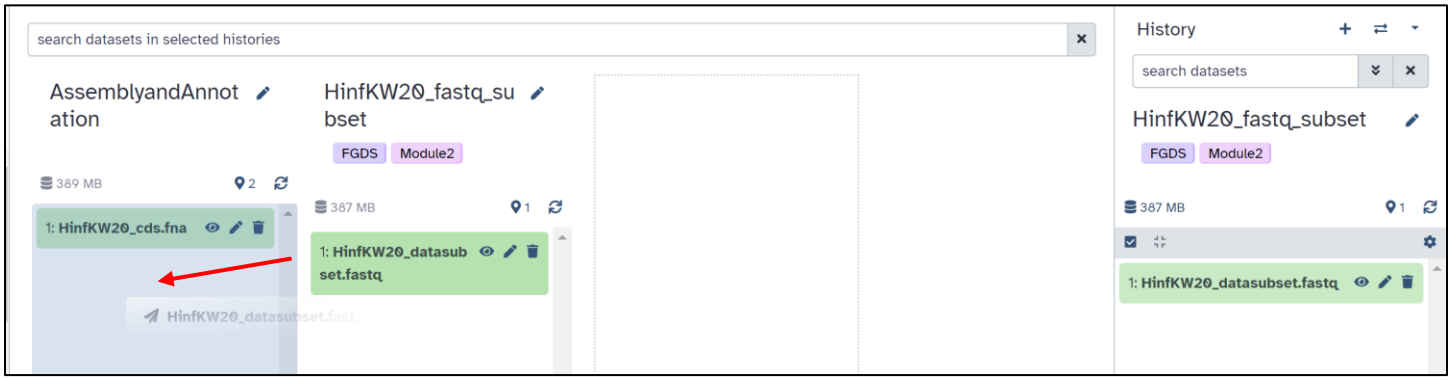
Now that we've imported a history into our instance of Galaxy we want to add that to our current **AssemblyandAnnotation** history.

2.4.1 Click on the **History drop down menu** (far right, three lines) and select **Show Histories Side-by-Side.** Alternatively, use the **History Multiview** icon on the *activity pane*. This view will provide a fast way to switch between multiple histories. You may initially see only our **AssemblyandAnnotation** history. Click on the secondary pane to select additional histories. Now we can see the newly imported history as well as our **AssemblyandAnnotation** history.



2.4.2 Ensure the **AssemblyandAnnotation** history is active in the History pane. If not, click on the **Switch to** button located below the history. The button will now read as **Current History**.

2.4.3 In the **HinfKW20_fastq_subset**, select the **HinfKW20_datasubset.fastq** and add it to the **AssemblyandAnnotation** history by *dragging and dropping* it into the AssemblyandAnnotation box.

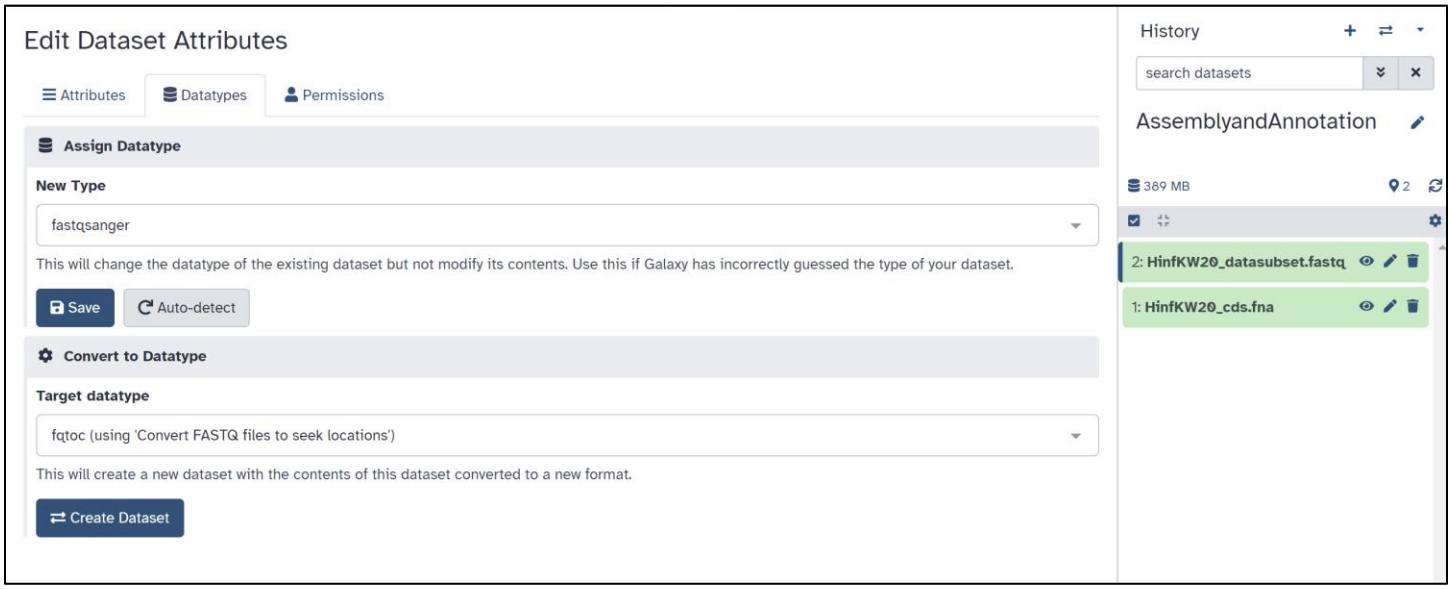


2.4.4 Return to the Galaxy homepage by clicking on the top left corner.

University of Toronto Centre for the Analysis of Genome Evolution & Function (cagef.utoronto.ca) Licensed under CC BY-NC-SA

2.4.5 You will now see the **HinfKW20_datasubset.fastq** file in your "History" panel. Verify that the datatype is **fastqsanger** using the *Edit Attributes* (pencil) key and clicking on the *Datatypes* tab.

2.4.6 Change datatype to **fastqsanger** if necessary.



Now we're prepared to begin working with our raw fastq data! We are using a single fastq file which we will treat as single-end sequencing data. However, a close inspection will reveal that the reads are indeed paired-end reads that have been combined!

Protip: view only the histories you want! As you work more and more with galaxy, you might find that you have too many histories to view. It's best to just look at the relevant histories for your work and you can control that with the **Select histories** button (far left) on the History Multiview.

It's also worth considering, in the naming of your histories, to include a date or additional tag that will help you differentiate different projects or datasets that you're trying to work with!

Under the activity pane **> Histories** you can retrieve a list of your histories where you can add tags to individual sets to make future searches easier!

University of Toronto Centre for the Analysis of Genome Evolution & Function (cagef.utoronto.ca) Licensed under CC BY-NC-SA

3.0.0 Quality filtering raw sequencing reads

Now that our data subset is available in our history, the first step is to perform a quality inspection with **FastQC** to identify potential issues with our data. After quality inspection, we will perform quality control with **Trimmomatic** (if necessary) to remove any of the identified issues.

3.1.0 Run *FastQC* on raw sequencing reads

The FastQC tool is a popular and helpful way to check the overall read quality of your dataset. While some sequencing platforms like Illumina will perform a set of quality filtering as they run, these are really just eliminating reads with a low passing filter. This means, as a whole the reads show quality issues that make their overall quality questionable. The FastQC tool takes this a step further to look closely at the Q-scores in a number of modules to create a human-readable html report.

3.1.1 From the Galaxy *activity pane* select [redacted]

3.1.2 Scroll down to review the purpose and input/output information for *FastQC*.

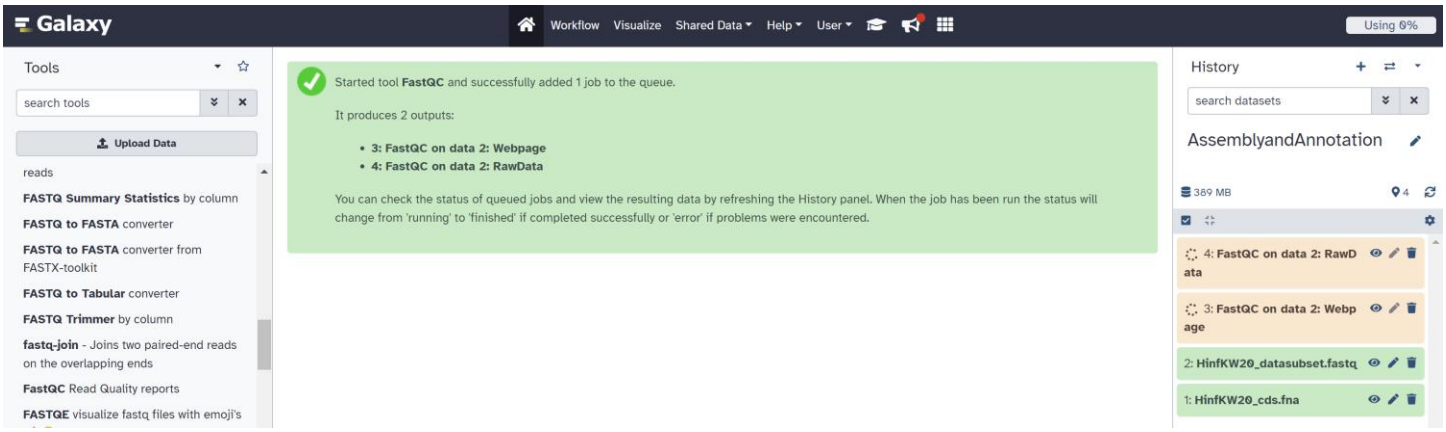
3.1.3 Execute *FastQC* with the following settings:

- Short read data from your current history: **2: HinfKW20_datasubset.fastq**
- Other options: **default**
- Use the **Run Tool** button (located at the top right **OR** below the tool options) to initiate **FASTQC**.

The run will generate two new history entries:

- **3: FastQC on data 2: Webpage**
- **4: FastQC on data 2: RawData**

Note that in some versions, you may be given the option to title your output. Our Galaxy instance uses the history value (***data 2***) as the naming scheme to produce output.



3.2.0 Reviewing your *FastQC* output

Once the background of the output files is green, the analysis is complete. We can review the results of the FastQC analysis on the *History pane*.

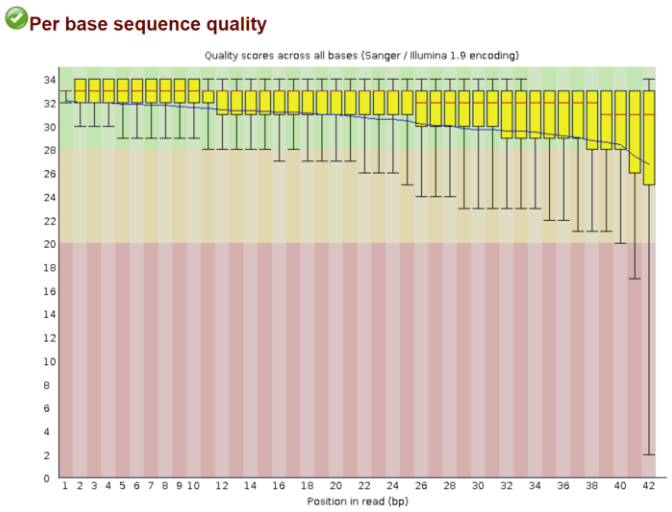
3.2.1 Review the HTML output from [redacted] by selecting the *Display* icon (eye) on the history entry.

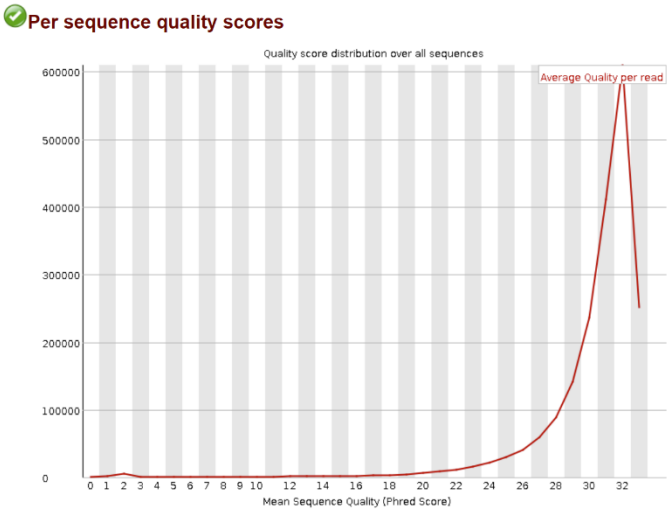
University of Toronto Centre for the Analysis of Genome Evolution & Function (cagef.utoronto.ca) Licensed under CC BY-NC-SA

3.2.2 You will see the following result sections that you can review:

- Basic Statistics (Pass)
- Per base sequence quality (Pass)
- Per tile sequence quality (Pass)
- Per sequence quality scores (Pass)
- Per base sequence content (Warning)
- Per base GC content (Pass)
- Per sequence GC content (Pass)
- Per base N Content (Pass)
- Sequence Length Distribution (Pass)
- Sequence Duplication Levels (Warning)
- Overrepresented Sequences (Pass)
- Adapter Content (Pass)

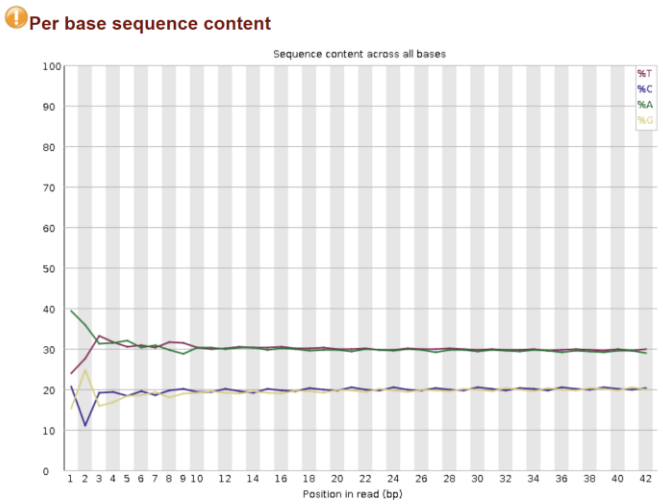
3.2.3 From our analysis we can see that overall, the read quality of along the set of raw reads remains quite high throughout the length of the reads. Even towards the tail end of the reads, the median Q score remains above 28. The average read quality per sequence is ~32.5 and the distribution from 2M reads shows bulk of reads to have a mean Q score above 28.

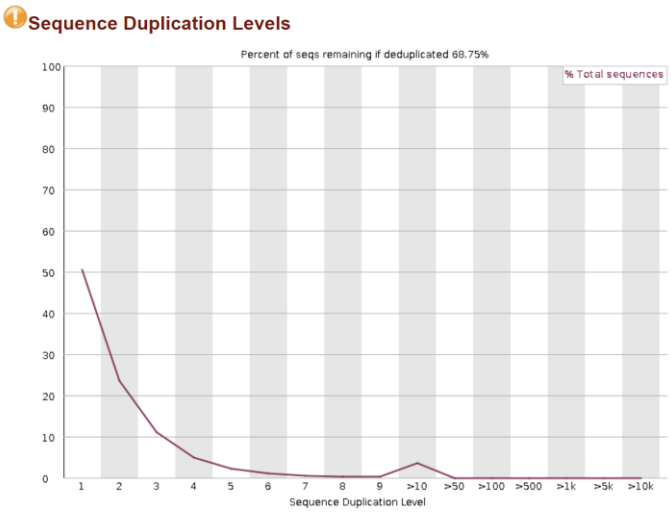




University of Toronto Centre for the Analysis of Genome Evolution & Function (cagef.utoronto.ca) Licensed under CC BY-NC-SA

3.2.4 While this analysis reveals that there are no major problems with our dataset, the sequence content is slightly biased at the start of every read, and there is a partial excess of duplicated sequences (~31%):





An alternative to FastQC: depending on your needs, experimental data types and level of throughput there are other programs that you may use to investigate your read quality. MultiQC is a program that we'll explore in later modules and it will aggregate results from multiple bioinformatics analyses into a single report. Other options for FASTQ file analysis include quack and fastp.

3.3.0 Use *Trimmomatic* to remove poor-quality reads

Trimmomatic performs a variety of useful tasks to help remove unwanted sequences from your reads. Developed for Illumina sequencing reads, this tool can remove adapter sequences generated during library preparation, while also removing sequences from the ends of your reads based on criteria like quality score.

Using this tool on our dataset may help remove some of the warnings received during our *FastQC* analysis. However, care should always be taken when trimming or removing reads, because strict filters can result in a lot of lost data and create new issues with sequencing reads.

3.3.1

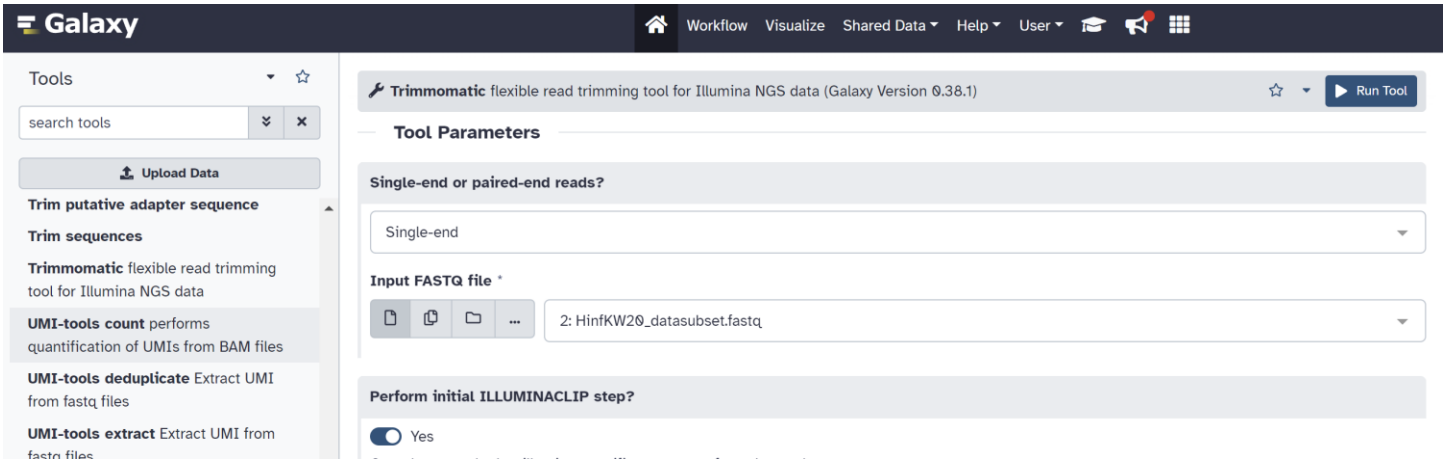
3.3.2 Review the parameters that can be used for *Trimmomatic* and the required inputs. Some commonly employed parameters include:

- **ILLUMINACLIP:** Trim adaptor and other Illumina sequences off the ends of reads.
- **SLIDINGWINDOW:** Trim reads based on average quality in a window of defined size.
- **MINLEN:** Drop all reads below a defined length.
- **LEADING/TRAILING:** Cut bases at the start/end of reads if below defined quality.
- **CROP/HEADCROP:** Cut a defined number of bases at start/end of reads (regardless of quality).
- **AVGQUAL:** Drop all reads below a defined average quality.

University of Toronto Centre for the Analysis of Genome Evolution & Function (cagef.utoronto.ca) Licensed under CC BY-NC-SA

3.3.3 Execute Trimmomatic with the following settings:

- Single-end or paired-end reads: **Single-end**
- Input FASTQ file: **2: HinfKW20_datasubset.fastq**
- Perform initial ILLUMINACLIP step? **Yes** (generates the following default values)
    - Select standard adapter sequences or provide custom? **Standard**
    - Adapter sequences to use: **TruSeq2 (paired-ended for Illumina GAII)**
    - Max mismatch count: **2**
    - Min matches between two 'adapter ligated' reads for PE palindrome read alignment: **30**
    - Min matches between adapter sequence and a read: **10**
    - Min adapter length that needs to be detected: **8**
- Trimmomatic Operation:
    - **Cut the specified number of bases from the start of the read (HEADCROP)**
        - Number of bases to remove from the start of the read: **10**
- Insert Trimmomatic Operation:
    - **Sliding window trimming (SLIDINGWINDOW)** with default values
        - Number of bases to average across: **4**
          Average quality required: **20**
- Insert Trimmomatic Operation:
    - [redacted]
    - [redacted]

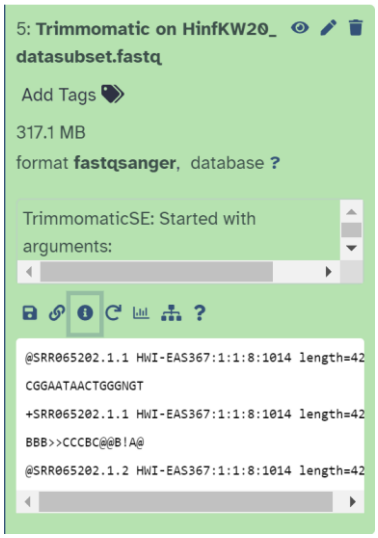


3.3.4 Note that with **Trimmomatic**, the order of operations matters. For example, if you performed the *SLIDINGWINDOW* trimming before using *CROP*, you would be cropping bases from the end of each read which have already been trimmed. This would not usually be what you want, because the *SLIDINGWINDOW* will already have removed many of the low-quality bases at the end of reads.

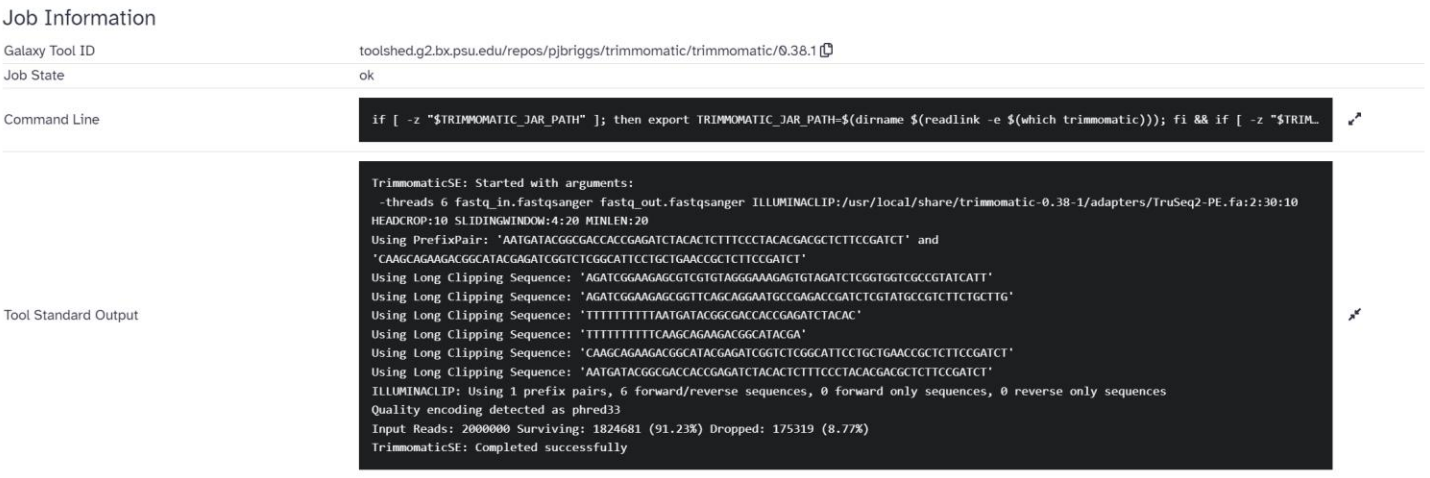
3.3.5 To view some of the run details, in the *History pane*, click on the entry for **Trimmomatic on HinfKW20_datasubset.fastq**. This will expand the history entry.

3.3.6 Click on the [redacted] which will bring up the running parameters of the *Trimmomatic* run.

University of Toronto Centre for the Analysis of Genome Evolution & Function (cagef.utoronto.ca) Licensed under CC BY-NC-SA



3.3.7 Under the *Job Information* section, find the **Tool Standard Output** entry and click on it to expand the information (far right, double-arrow icon).



3.3.8 Note that of 2M reads, 91.23% survived while 8.77% were dropped. The **standard output** (log file) information also includes the Illumina adapter sequences used in the clipping process.

3.4.0 Rerun your trimmed data in *FastQC*

Now that we've trimmed our data of adapters sequences and some poor-quality reads, we can return to examine how this changes our FastQC output.

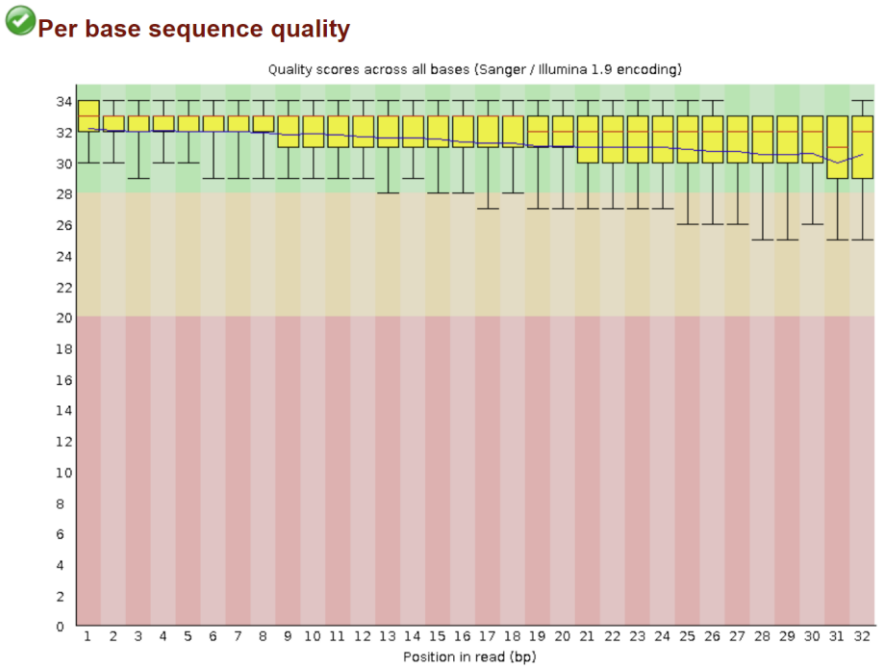
3.4.1 From the Galaxy *activity pane* select **Tools > FASTQ Quality Control > FastQC Read Quality reports**

3.4.2 Execute FastQC with the following settings:

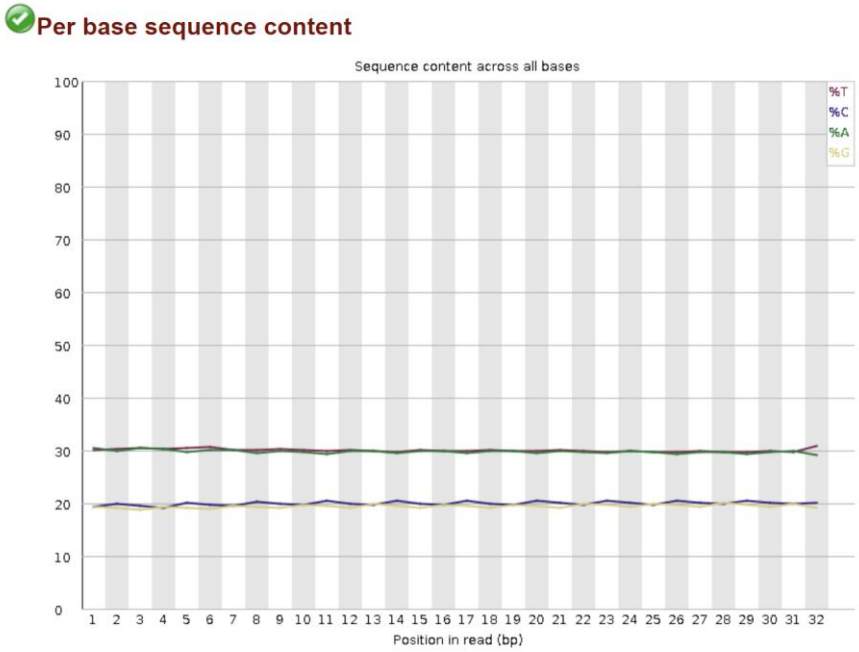
- Short read data from your current history: **5: Trimmomatic on HinfKW20_datasubset.fastq**
- Other options: **default**

3.4.3 Review the HTML output from **7: FastQC on data 5: Webpage** by selecting the *Display* icon (eye) on the history entry.

University of Toronto Centre for the Analysis of Genome Evolution & Function (cagef.utoronto.ca) Licensed under CC BY-NC-SA



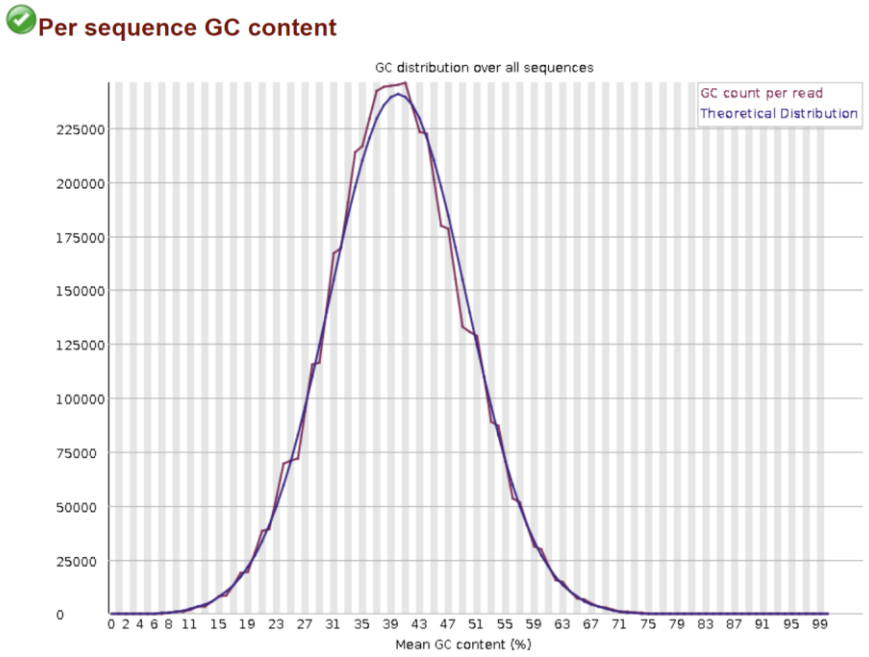
- We have solved our initial issues with **Per base sequence content** through the **HEADCROP** command which removed the first 10 bases of each read thus reducing the initial biases at the start of our reads.



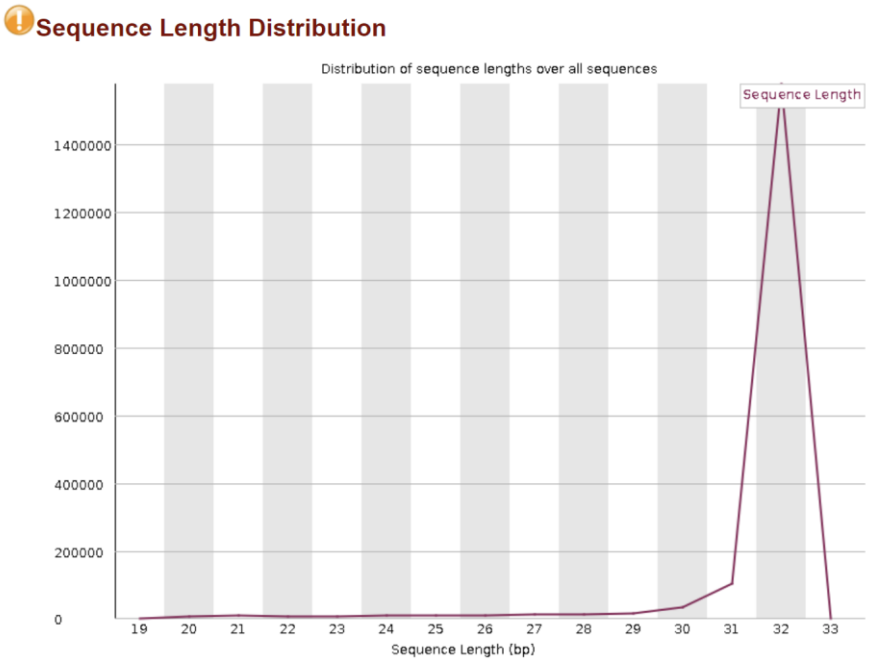
- Check out the per sequence quality scores distribution as well. While not pictured here, take a close look at the x-axis values. What do you think is the cause of this?

- As a consequence, however, our reads are now at most only 32 bases. Looking at the **Per sequence GC content** we can see that our trimming and cleaning has not altered the distribution. This is likely due to our removal of smaller fragments below 20bp.

University of Toronto Centre for the Analysis of Genome Evolution & Function (cagef.utoronto.ca) Licensed under CC BY-NC-SA



- We can also see from our **Sequence Length Distribution** that our reads are no longer strictly 42 basepairs but range from 20 to 32 basespairs in length.



Check out the raw data: When we ran the FastQC on our output, it generated 2 sets of data output: Webpage and RawData. Depending on your needs, you can actually reference the raw data used to generate these visualizations. This can be used to give you a better idea, for instance, of the range on your Phred Scores across each base!

University of Toronto Centre for the Analysis of Genome Evolution & Function (cagef.utoronto.ca) Licensed under CC BY-NC-SA

4.0.0 Genome Assembly with Velvet

To recap, we have imported raw sequence reads from the *Haemophilus influenza* genome and have trimmed and filtered these reads using a combination of ***Trimmomatic*** and ***FastQC*** to analyse the read quality and distribution. We are now ready to assemble these reads into our first draft genome assembly with the Velvet tool.

Given the nature of our reads now ranging in size from 20 to 32 basepairs we can assume that we will end up with several smaller contigs (assembled fragments). Other contributors of small contigs can be low coverage or repetitive regions. Regardless, we'll have an assembly that we can begin to annotate in later steps.

The ***Velvet*** assembly tool runs in two steps:

1) Use ***velveth*** to build a ***h***ash table of k-mers in each read ie. it tracks each read by its different subsequences.

2) Use ***velvetg*** to perform a ***g***enome assembly based on the hash table from ***velveth***

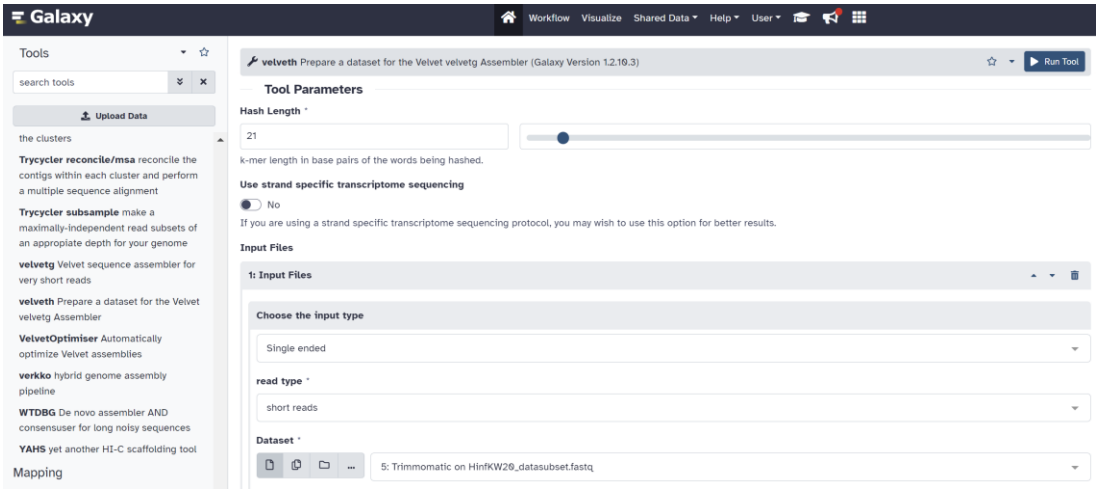
4.1.0 Use *velveth* to produce a hash table of sequences

4.1.1

4.1.2 Scroll down to review the purpose and input/output information about ***velveth***.

4.1.3 Execute ***velveth*** to create your hash table using the following settings:

- Hash Length: **21**
- Use strand specific transcriptome sequencing: **No**
- **Insert Input Files >** Input Files:
    - Choose the input type: **Single ended**
    - Read type: **short reads**
    - Dataset: **5: Trimmomatic on HinfKW20_datasubset.fastq**



4.2.0 Use *velvetg* to produce a genome assembly

Now that we have our hash table from ***velveth*** we can send this on to ***velvetg*** which will use the information to construct a de Bruijn graph by adding the k-mers one-by-one to the graph and then chaining and merging paths and nodes to produce the final contig sequences.

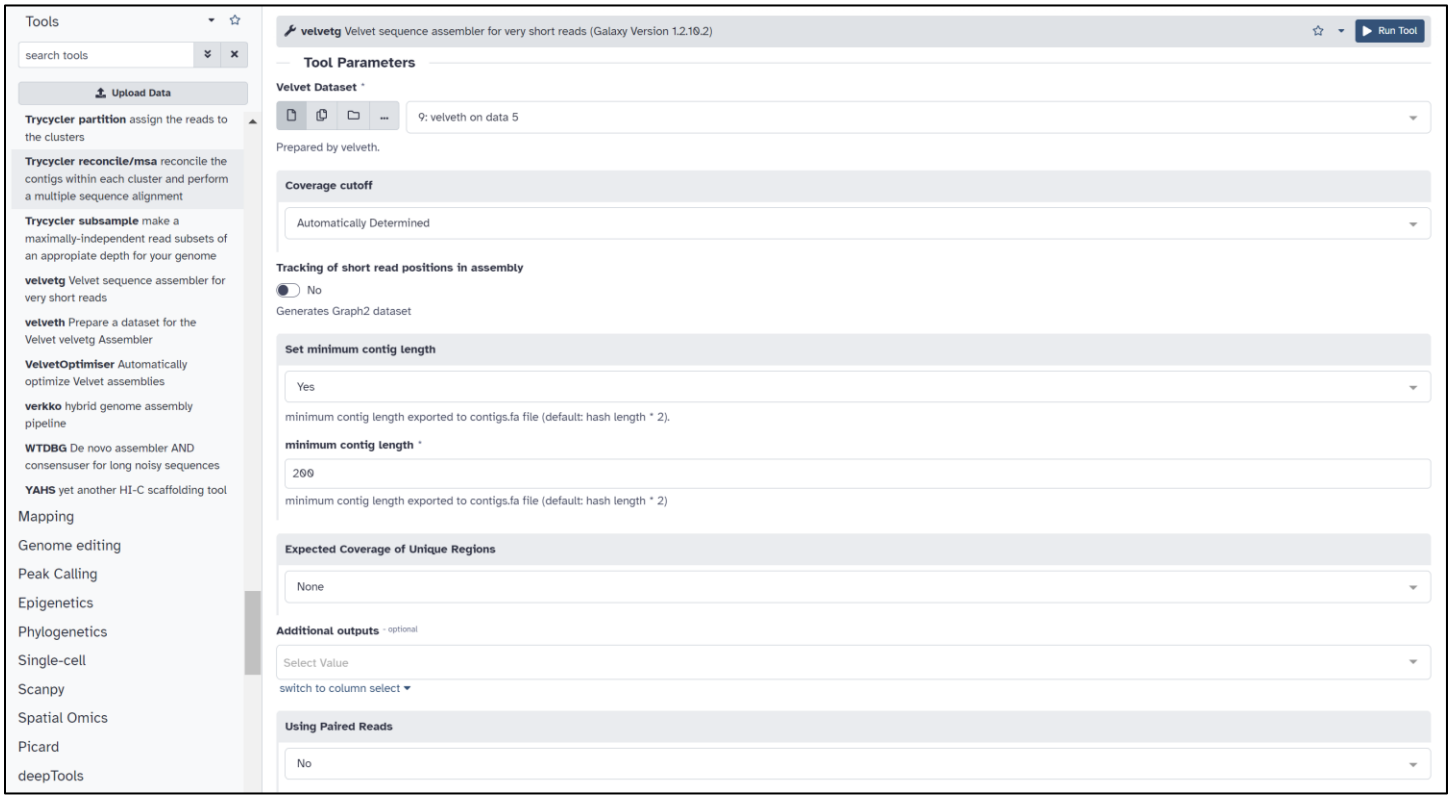
University of Toronto Centre for the Analysis of Genome Evolution & Function (cagef.utoronto.ca) Licensed under CC BY-NC-SA

4.2.1 [redacted]

4.2.2 Scroll down to review the purpose and input/output information about ***velvetg***.

4.2.3 Execute ***velvetg*** to produce your assembly using the following settings:

- Velvet Dataset: **velveth on data 5**
- Coverage cutoff: **Automatically Determined**
- Tracking of short read positions in assembly: **No**
- Set minimum contig length: **Yes**
    - Minimum contig length: **200**
- Expected Coverage of Unique Regions: **None**
- Additional outputs: **leave this blank** but it will still secretly select
    - Generate a AMOS.afg file
    - Generate a UnusedReads fasta file
    - Generate a velvet LastGraph file
- Using Paired Reads: **No**



What is a de Bruijn graph? Yes, you have generated a genome using **velvetg** but how does it *actually* work? All the **k-mers** are represented in a graph with connected neighbours being **k-mers** overlapping by (**k-1)** bases. Each node stores information about how often its k-mer is seen within the data. For more information on using the Velvet *de novo* assembler, check out Zerbino 2010 on Pubmed

Here's a simple video to help get you started on the concept:
https://youtu.be/OY9Q_rUCGDw?si=eur9rq0a3EFAcGVP

University of Toronto Centre for the Analysis of Genome Evolution & Function (cagef.utoronto.ca) Licensed under CC BY-NC-SA

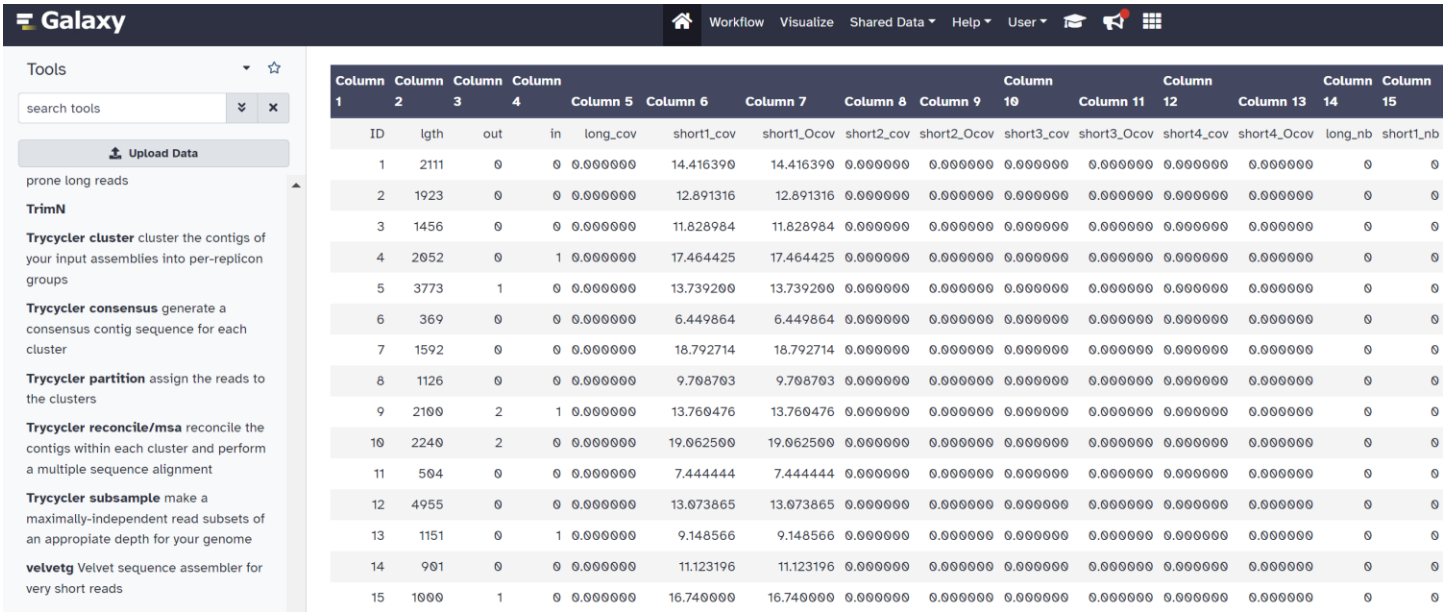
4.3.0 Reviewing your velvetg results

Looking at our *History pane*, we see that *velvetg* has generated five sets of output but we are just interested in two: **velvetg on data 8: Contigs** and **velvetg on data 8: Stats**.

4.3.1 The *velvetg on data 8: Contigs* entry in our history contains the final contigs generated by our assembly. Under *Dataset Details* > **Job Information > Tool Standard Output**, you can review summary information generated by the tool.

You will be able to find an N50 and total length for your assembly at the bottom of file, but these values are for your de Bruijn graph so they don't account for contig filtering. N50 is a quality metric that is defined as the shortest sequence length in the set of largest contigs that sum to 50% of the genome length. Higher N50 values mean that you have a more contiguous assembly.

4.3.2 Review the **Stats** output from *velvetg on data 8: Stats > View*.



Column 1	Column 2	Column 3	Column 4	Column 5	Column 6	Column 7	Column 8	Column 9	Column 10	Column 11	Column 12	Column 13	Column 14	Column 15
ID	lgth	out	in	long_cov	short1_cov	short1_Ocov	short2_cov	short2_Ocov	short3_cov	short3_Ocov	short4_cov	short4_Ocov	long_nb	short1_nb
1	2111	0	0	0.000000	14.416390	14.416390	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0	0
2	1923	0	0	0.000000	12.891316	12.891316	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0	0
3	1456	0	0	0.000000	11.828984	11.828984	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0	0
4	2052	0	1	0.000000	17.464425	17.464425	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0	0
5	3773	1	0	0.000000	13.739200	13.739200	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0	0
6	369	0	0	0.000000	6.449864	6.449864	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0	0
7	1592	0	0	0.000000	18.792714	18.792714	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0	0
8	1126	0	0	0.000000	9.708703	9.708703	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0	0
9	2100	2	1	0.000000	13.760476	13.760476	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0	0
10	2240	2	0	0.000000	19.062500	19.062500	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0	0
11	504	0	0	0.000000	7.444444	7.444444	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0	0
12	4955	0	0	0.000000	13.073865	13.073865	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0	0
13	1151	0	1	0.000000	9.148566	9.148566	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0	0
14	901	0	0	0.000000	11.123196	11.123196	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0	0
15	1000	1	0	0.000000	16.740000	16.740000	0.000000	0.000000	0.000000	0.000000	0.000000	0.000000	0	0

Now that we have our genome assembled, and a list of contigs, what should we do with it? Annotate it of course!

Quality metrics for your genome assembly will often include N50 and L50. If you have a sense of how big your genome should be (ie through experimental/empirical studies) this will give you an idea of 1) how close your genome resembles a complete one; 2) How many potential gaps/problem areas may exist in your genome.

The N50 is calculated by summing contig lengths in descending order until they are equal to 50% of the sum total contig length. The smallest contig in this set is the N50 value. The L50, on the other hand, represents the total number of contigs used to calculate the N50.

An additional metric, the NG50, follows the same method as N50 except it substitutes the estimated genome length for the sum total contig length (see Bradnam et al., Gigascience 2013)

University of Toronto Centre for the Analysis of Genome Evolution & Function (cagef.utoronto.ca) Licensed under CC BY-NC-SA

4.4.0 Determining the quality of your assembly

Now that we've generated an assembly, you may be wondering, "Is it any good?". How well would this compare to the genome assembly for *H. influenza* that we downloaded for **Module 1**? There are some genome assembly analysis tools that you can use to compare the results from various genome assemblers or variations from the same assembler. Today we'll use a **QU**ality **AS**sessment **T**ool, **QUAST**, to help us compare our *velvet* assembly to a reference genome.

4.4.1 First we'll retrieve a copy of our *H. influenzae* genome with *activity pane* **> Upload.**

4.4.2 **Choose local file** and upload the file [redacted] from your computer.

4.4.3 From the Galaxy *activity pane* **> Tools > Assembly > Quast Genome assembly Quality**.

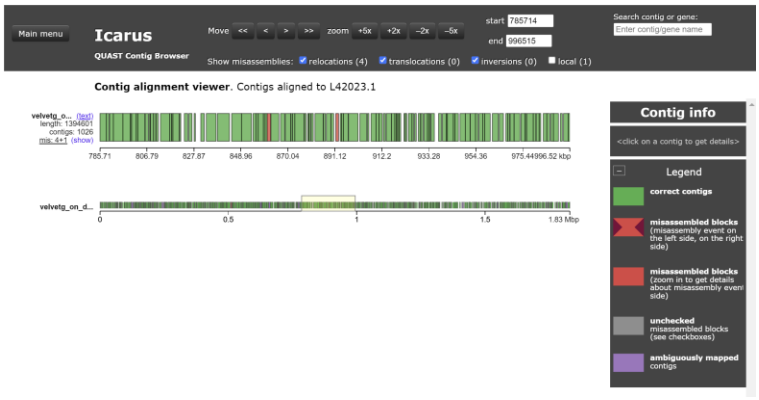
4.4.4 Scroll down to review the purpose and input/output information about *QUAST*.

4.4.5 Execute *QUAST* to evaluate your *velvetg* assembly using the following settings:

- Assembly mode? **Individual assembly (1 contig file per sample)**
- Use customized names for the input files? **Yes, specify custom names**
    - Contig/scaffolds file: **velvetg on data 8: Contigs**
    - Name: [redacted]
- Type of assembly: **Genome**
- Use a reference genome: **Yes**
- Reference genome: **HinfKW20_genomic.fna**
- Output files:
    - [redacted]
    - [redacted]
- Other options: **default**

The QUAST program will produce a series of 4 files:

- **Tabular report**: Various summaries of your assembly quality, including the alignment size.
- **HTML report**: An interactive report of your assembly quality which includes a link to *View in Icarus contig browser* which will take you to a mini genome viewer to examine contig alignments.
    - A visualization of *where* contigs align to your reference genome. This is also an interactive browser so you can zoom and examine individual contigs and their alignment.
- **Misassemblies report**: A table outlining potential problems with the assembly including misassembled contigs.
- **Unaligned contigs report**: A report on the unaligned contigs from the assembly or assemblies.



The Icarus alignment viewer shows the placement of our assembly contigs

University of Toronto Centre for the Analysis of Genome Evolution & Function (cagef.utoronto.ca) Licensed under CC BY-NC-SA

4.5.0 Examining your contigs for consistency in the QUAST alignment view

Looking at the output from our QUAST analysis, one of the helpful visual tools is the alignment view report. This graphical interface provides the user with a more detailed view of misassembled contigs based on the reference sequence.

4.5.1

4.5.2



4.5.2 Use the bottom viewer window viewer to slide over to about 0.5 Mbp. While hovering on the left and right side of this (yellow) window click and drag the edges to decrease the window size. Your view on the above **contig viewer** should zoom in correspondingly. Alternatively, use the available navigation buttons in the *Icarus* top ribbon bar.

4.5.3 In the upper viewer, select the red contig at **510.8 kbp**. This will provide contig information to you on the right-side pane.

4.5.4 Looking at the block information, you can see that there are 2 blocks listed for this contig. You can click on the "+" sign to expand these.

The expanded block information denotes the alignment location of sections from the same contig. This contig is 3151 bp in length but is split between locations at 510,839 (2897 bp) and 1,163,665 (254 bp). This suggests that 1) this contig may have been incorrectly constructed/merged between two smaller contigs; 2) the original genome sample contained a potential translocation; or 3) the original library preparation resulted in the concatenation of unrelated sequences. In any case, understanding a misassembled contig like this would require further exploration.

4.5.5 Looking again in the same area, you may notice a purple block near 659 kbp. Click on this block to examine information about it.

The Contig info for this block should show a contig of about 3430 bp. It is ambiguously mapped to 6 positions in the reference genome. The lower full-genome map will now also show many small arrows identifying the specific regions where this contig has ambiguously mapped.

Genome assemblies can be complex! Given that we are working with a small genome, the complexity of our assembly is relatively low. You can see that our assembly didn't quite cover the entire reference genome and there are a few possible reasons for why that might be the case. We'll explore some of those in this week's assignment and next week's module as well.

The quality of your genome assembly will depend quite a bit on factors like read length, read depth, single vs. paired-end sequencing format, and is best complemented through additional sequencing methods like long-read sequencing and Hi-C sequencing. These methods can help reduce gaps, and correct contig orientation respectively.

University of Toronto Centre for the Analysis of Genome Evolution & Function (cagef.utoronto.ca) Licensed under CC BY-NC-SA

5.0.0 Genome annotation

As is the case with most forms of genome analysis, there are many options available for genome annotation - the process of identifying coding regions in a genome and assigning a function to them.

5.1.0 Genome annotation with Prokka

Prokka is designed for smaller genomes like those of bacteria, archaea, and viruses. Its major advantages are speed and simplicity. **Prokka** can fully annotate a typical 4Mbp genome in less than 10 minutes on most modern personal computers. Furthermore, it integrates coding sequence identification and functional assignment into a single command, which makes it very easy to use.

For eukaryotic genomes, you must consider introns and exons for each gene. It is also important that you mask repetitive sequences in these genomes. Some common programs for the annotation of eukaryotic genomes include **Augustus**, **Maker**, and **Pasa**. Information about these programs is available online.

5.1.1 Go to *activity pane* **> Tools > Annotation > Prokka Prokaryotic genome annotation**

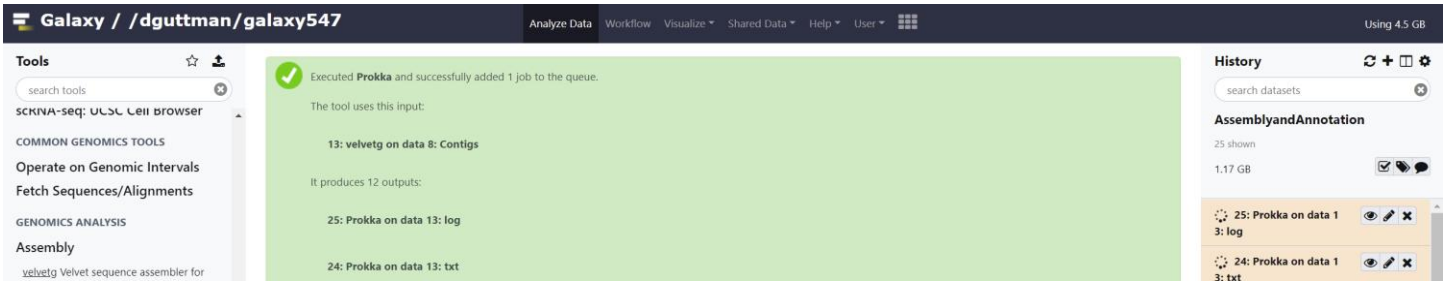
5.1.2 Scroll down to review the purpose and input/output information about Prokka.

There are lots of options here that can be useful depending on the goals of the annotation. These include, for example, specifying genus or species-specific databases to prioritize when assigning functions to coding regions. You can also specify which output files you want **Prokka** to generate. Today, we will simply be running **Prokka** using default settings and will be assigning functions with the default BLAST database.

5.1.3 Execute Prokka to produce your annotation using the following settings:



5.1.4 Scrolling past the remaining parameters (leave as **default**) we find the *Additional outputs* section where we can choose which files to create from the annotation. Leave these as the default (**Select all**).



5.2.0 Review your *Prokka* output

You'll see that our *Prokka* execution has generated 12 files, many of which should appear familiar to you from module 1: **gff**, **gbk**, **fna**, **faa**, **ffn**, **sqn**, **fsa**, **tbl**. In addition to this it has also produced a coding sequence (**tsv**); a discrepancy report (**err**); a brief summary (**txt**); and a more verbose run report (**log**).

Many of these files are what we would find accompanying our assembly report from NCBI.

University of Toronto Centre for the Analysis of Genome Evolution & Function (cagef.utoronto.ca) Licensed under CC BY-NC-SA

6.0.0 Downloading data and saving your pipeline

Downloading output files and applying appropriate naming conventions from Galaxy analyses is not as easy as we might like. This process can be sped up somewhat by gaining familiarity with the command line, but in this section of the course we will stick with **Finder** and **File Explorer**.

6.1.0 Create a dataset list of your data from the *History pane*

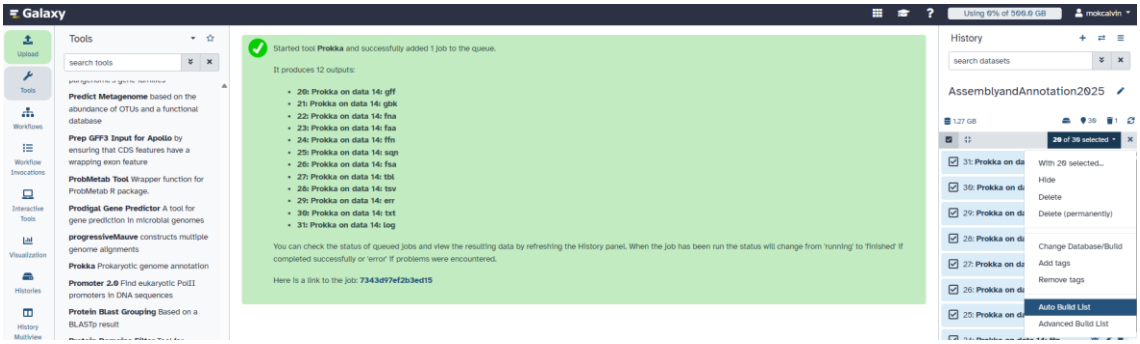
6.1.1 Recall that you have already set up a subfolder for this module under your main FGDS folder.

6.1.2 In the Galaxy *History pane* on the top-left corner of the history, select the *Select items* (checkbox) button. You'll see selectable checkboxes are now available for each history entry.

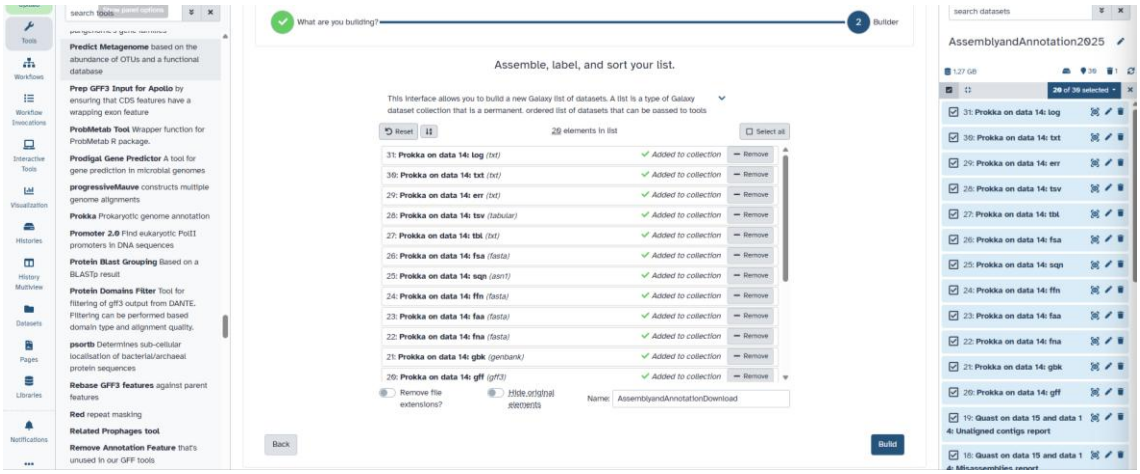
6.1.3 Select the following history entries to download:



6.1.4 From the *History pane* select the *20 of 30 selected* dropdown menu and select **Auto Build List**



6.1.5 *Deselect* **Remove file extensions?** and **Hide original elements** and name the list **AssemblyandAnnotationDownload** before selecting *Build*. You will now see a new entry in the *History pane* with the name of your newly created list.



University of Toronto Centre for the Analysis of Genome Evolution & Function (cagef.utoronto.ca) Licensed under CC BY-NC-SA

6.2.0 Download your dataset list (Complete this Post-lecture)

Now that we've created a list of 20 items we can download it to your own desktop.

6.2.1 Go to the *History pane* and deselect the *Select items* checkbox icon (if needed) and then select the **AssemblyandAnnotationDownload** entry. This will show the 20 items of your list.

6.2.2 From the *History pane* click on the *Download* button (top left of history item list) and save the file to your **~/FGDS/Module2/downloads** folder. Notice that a single **.zip** compressed file will download to your directory.

6.2.3 Decompress the **AssemblyandAnnotationDownload.zip** into a folder using your favourite program. 7zip is a good open source Windows file archiver and MacOS should come with a native program for opening these files. You may also be able to navigate the archive directly in a file explorer such as in Windows 11.

6.2.4 Unarchive **AssemblyandAnnotationDownload.tar** and place its contents into **~/FGDS/Module2/** and rename the files (and folders) as follows:

Download name	New name
FastQC on data 2: Webpage.html	FastQC_RawOutput.html
FastQC on data 2: Webpage	FastQC_RawOutput
FastQC on data 5: Webpage.html	FastQC_Trimmed.html
FastQC on data 5: Webpage	FastQC_Trimmed
Prokka on data x: err.txt	HinfKW20_prokka.err
Prokka on data x: faa.fasta	HinfKW20_prokka.faa
Prokka on data x: ffn.fasta	HinfKW20_prokka.ffn
Prokka on data x: fna.fasta	HinfKW20_prokka.fna
Prokka on data x: fsa.fasta	HinfKW20_prokka.fsa
Prokka on data x: gbk.genbank	HinfKW20_prokka.gbk
Prokka on data x: gff.gff3	HinfKW20_prokka.gff3
Prokka on data x: log.txt	HinfKW20_prokka.log
Prokka on data x: sqn.asn1	HinfKW20_prokka.sqn.asn1
Prokka on data x: tbl.txt	HinfKW20_prokka.tbl
Prokka on data x: tsv.tabular	HinfKW20_prokka_cds.tsv
Prokka on data x: txt.txt	HinfKW20_prokka.txt
Quast on data x and data y: HTML report.html	HinfKW20_quast_report.html
Quast on data x and data y: HTML report	HinfKW20_quast_report
Quast on data x and data y: Misassemblies report.tabular	HinfKW20_quast_misassemblies.tabular
Quast on data x and data y: tabular report.tabular	HinfKW20_quast_report.tabular

University of Toronto Centre for the Analysis of Genome Evolution & Function (cagef.utoronto.ca) Licensed under CC BY-NC-SA

Quast on data x and data y: Unaligned contigs report.tabular	HinfKW20_quast_unalignedContigs.tabular
velvet on data 8: Contigs.fasta	HinfKW20_velvetContigs.fasta
velvet on data 8: Stats.tabular	HinfKW20_velvetStats.tsv

6.3.0 Save your history as a workflow



Now that you've saved your files it's time to convert your history (which is also saved in Galaxy) into a workflow. A workflow is the equivalent of a bioinformatic pipeline – a series of commands that will run on your data to produce a final output.

Saving workflows will allow you to perform the same analyses that we just performed on a completely different dataset. For example, if there was a second *H. influenzae* strain that you had sequenced, you could run the same analysis we just ran on the *H. influenzae* KW20 strain with the same parameters in a single step. This is important for both efficiency and reproducibility.

6.3.1 Go to the *History pane* and select the *History settings* button (gear), if your original datasets have become hidden.

6.3.2 From the **With entire history** drop down menu, select **Unhide all hidden content** and confirm.

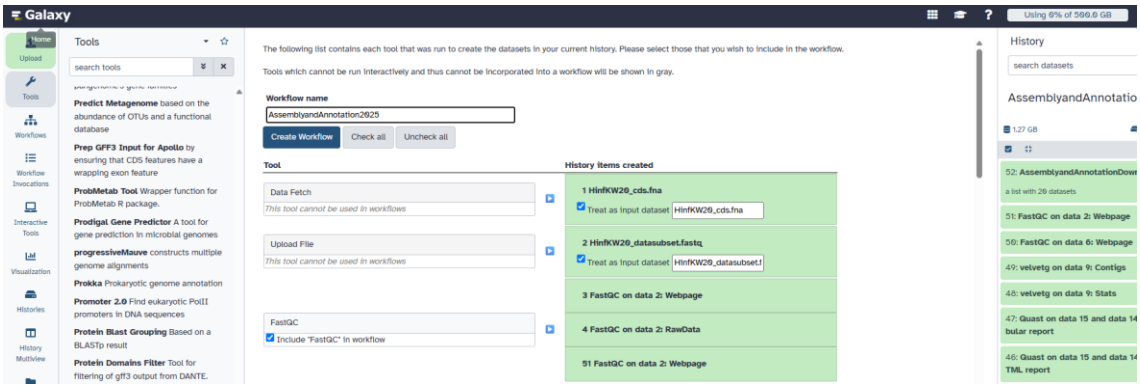
6.3.3 Go to the *History pane* and select the *History options* button.

6.3.4 Under the *History options* dropdown (3 lines, top right) select

Notice that the *Data Fetch* and *Upload File* tools include a description "This tool cannot be used in workflows". These will inevitably be removed from the workflow when we create it.

6.3.5 Simplify the *Workflow name* to **AssemblyandAnnotation** in the upper left corner.

University of Toronto Centre for the Analysis of Genome Evolution & Function (cagef.utoronto.ca) Licensed under CC BY-NC-SA

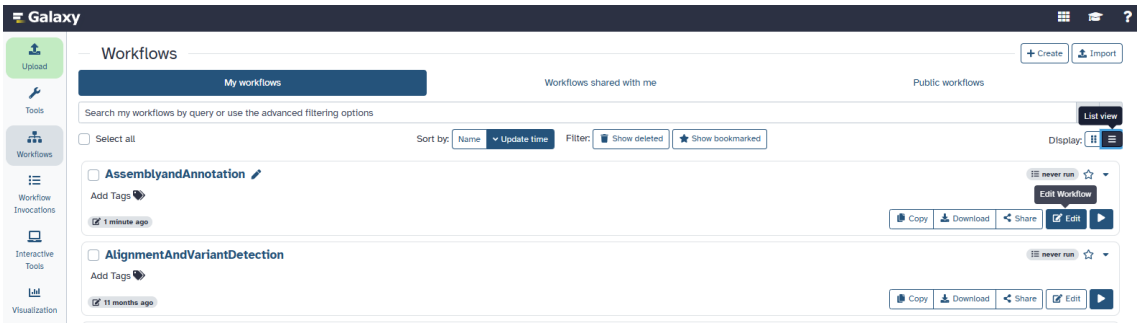


6.3.6 Select the ***Create Workflow*** button.

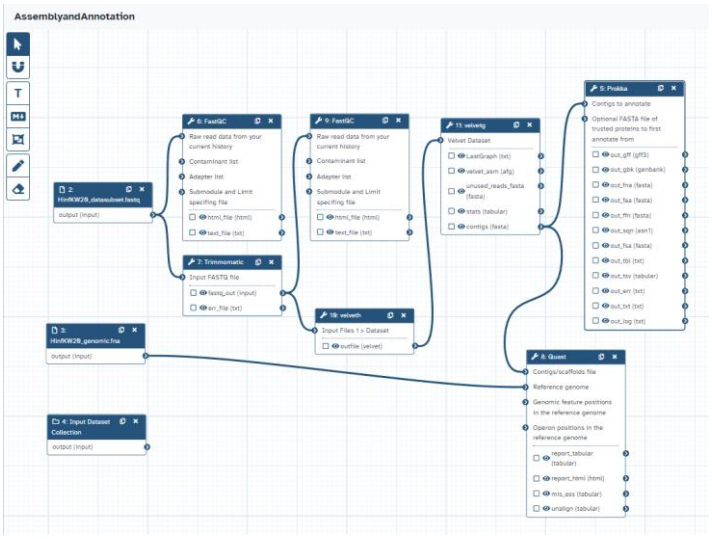
6.4.0 Editing your workflow

Now that a workflow has been created you can edit and rearrange portions as you see fit. You can even alter some of the parameters, add/remove steps, specify the files you want to include as output and resave the workflow or save it as a completely new one.

6.4.1 On the Galaxy ***activity pane*** select ***Workflow***



6.4.2 On the Workflows page find the ***AssemblyandAnnotation*** workflow and select the [redacted] This will load the workflow editor as follows.



6.4.3 Use the visual editor to re-arrange the steps into a clear set of analyses. This will make it easier to see how you might want to modify the input/output files. Notice how the tools are connecting the output of one with the input of another.

University of Toronto Centre for the Analysis of Genome Evolution & Function (cagef.utoronto.ca) Licensed under CC BY-NC-SA

6.4.4 If needs, connect the contigs (fasta) output from velvetg to the *Prokka* **Contigs to annotate** input and the *Quast* **Contigs/scaffolds file** input as seen on the right-hand side of the diagram in 6.4.2.

6.4.5 In the right-hand *Details pane* you can see the specific parameters set for each tool when you click on the individual tool information boxes.

Notice that in this pane you can also label and annotate the individual steps. This can be a useful way to comment on what you are doing and why.

6.4.6 Also in

In the *Attributes pane* you can now see additional tags and annotations you can add to the overall workflow. These are helpful attributes to add if you are sharing your workflow with others as it will make it searchable and descriptive. You can also see previous saved versions of this workflow in the *Version* dropdown menu.

6.4.7 Use the *Save* icon (floppy disk) to save your workflow.

6.4.8 Click on the Galaxy home icon in the top left corner.

6.5.0 Run your saved workflow

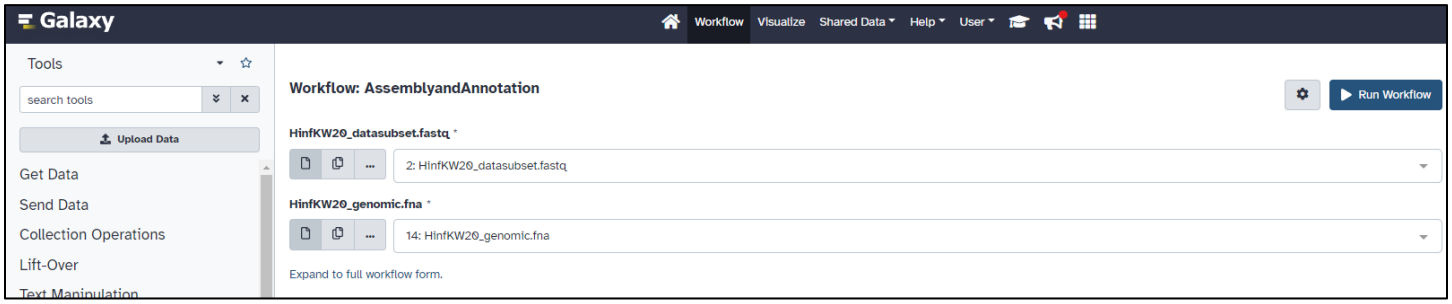
Now that we've saved our updated workflow, how do we apply this to another dataset? Recall that our saved workflow appears to already include a dataset.

6.5.1 On the Galaxy *activity pane* select *Workflows*

6.5.2 On the Workflow page find the entry for *AssemblyandAnnotation* and select the *Run workflow* option (far right). This will load the workflow in its run order as follows.

6.5.3 Under the *Workflow Run Settings* button (top-right gear icon), decide whether you'd like to **Send results to a new history** for this run or for it to go to your current history. If you choose **Yes**, then be sure to give your history a name after.

6.5.4 *Step 1* will ask you to enter an initial file for analysis. The dropdown menu will include any available fastq files that are present in your current *History pane*. It is here that you will choose a set of fastq reads that are available in your history.



6.5.5 Double check the settings/parameters across your tool. You'll see that they can be edited individually but if you are saving this as a workflow, you'll want to keep them constant for consistency. If you're experimenting with your settings, you should note they will not be saved and you should experiment with these in the **Workflow Edit** page instead.

University of Toronto Centre for the Analysis of Genome Evolution & Function (cagef.utoronto.ca) Licensed under CC BY-NC-SA

7.0.0 Class summary

That concludes our second lecture and introduction to working with the Galaxy platform. Next week we'll dig into some other tools on the platform to explore RNAseq. Altogether we've explored the following:

- Importing data files into Galaxy.
- Quality control analysis of raw and filtered reads with *FastQC*.
- Trimming and filter reads with *Trimmomatic*.
- Small genome assembly using *Velvet*.
- Quality analysis of genome assemblies using *QUAST*
- Prokaryotic genome annotation with *Prokka*.
- Downloading your datasets.
- Saving, editing, and running workflows based on your previous history.

7.1.0 Post-lecture assessment (9% of final grade)

Soon after this lecture, a homework assignment will be made available on Quercus in the assignment section. It will build on the ideas and/or data generated within this lecture. Each homework assignment will be worth 9% of your final mark. If you have assignment-related questions, please try the following steps in the order presented:

- Check the internet for a solution – read forums and learn to navigate for answers.
- Generate a discussion on Quercus outlining what you've tried so far and see if other students can contribute to a solution.
- Contact course teaching assistants or the instructor.

7.2.0 Suggested class preparation for Module 3

Next week we will begin exploring the Galaxy interface and learning how to perform quality control and genome assembly on data from sources like the SRA. To prepare for this, we suggest the following reading which will be available from the Module 3 section of Quercus:

- *Altmann et al., 2012, Human Genetics, 131:1541-1554.*
- *Yendrek et al. BMC Research Notes 2012, 5:506*
- *Kim et al., 2019, Nat. Biotech., 37:907-915*

7.3.0 Acknowledgements

This course was originally conceived and produced by Dr. David Guttman and Dr. Marcus Dillon.

- **Revision 1.0.0**: edited and prepared by David Guttman, Ph.D. and Marcus Dillon, Ph.D.
- **Revision 2.0.0**: edited and prepared for *CSB1021HF LEC0131, 10-2021* by Calvin Mok, Ph.D., Education and Outreach, CAGEF.
- **Revision 2.0.1**: edited and prepared for *CSB1021HF LEC0131, 10-2023* by Calvin Mok, Ph.D., Education and Outreach, CAGEF.
- **Revision 2.1.0**: edited and prepared for *CSB1021HF LEC0131, 10-2024* by Calvin Mok, Ph.D., Education and Outreach, CAGEF.
- **Revision 2.1.1**: edited and prepared for *CSB1021HF LEC0131, 10-2025* by Calvin Mok, Ph.D., Education and Outreach, CAGEF.
- **Revision 2.1.2**: edited and prepared for *CSB1021HF LEC0131, 10-2026* by Calvin Mok, Ph.D., Education and Outreach, CAGEF.

University of Toronto Centre for the Analysis of Genome Evolution & Function (cagef.utoronto.ca) Licensed under CC BY-NC-SA

7.4.0 References

- Andrews, S. (2010). *FastQC: A Quality Control Tool for High Throughput Sequence Data* [Online]. Available online at: http://www.bioinformatics.babraham.ac.uk/projects/fastqc/

- Bolger A.M., Lohse M., Usadel B. (Bioinformatics, 2014*). Trimmomatic: a flexible trimmer for Illumina sequence data.* DOI: 10.1093/bioinformatics/btu170

- Gurevich A., et al. (Bioinformatics, 2013). *QUAST: quality assessment tool for genome assemblies.* DOI: 10.1093/bioinformatics/btt086

- Seemann T. (Bioinformatics, 2010) *Prokka: rapid prokaryotic genome annotation.* DOI: 10.1093/bioinformatics/btu153

- Zerbino D.R., Birney E. (Genome Res. 2008). *Velvet: Algorithms for de novo short read assembly using de Bruijn graphs.* DOI: 10.1101/gr.074492.107



University of Toronto Centre for the Analysis of Genome Evolution & Function (cagef.utoronto.ca) Licensed under CC BY-NC-SA